

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A Knowledge Representation Design Assessment

Code of Conduct:

I ____Sk. Fawaz Ahamad / 192425390____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sk. Fawaz Ahamad

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	

Total	100%	
-------	------	--

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.

3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.

- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.

6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

SMART HEALTHCARE DIAGNOSIS SYSTEM

Knowledge Representation, Logical Inference and Rule-Based Reasoning A

Complete Academic Case Study and Prototype Report

Document Detail	Value
Student Name	Sk. Fawaz Ahamad
Programme	B. Tech – Artificial Intelligence & Machine Learning
Year / Semester	3rd Year
Institution	Saveetha Institute of Medical and Technical Sciences (SIMATS)
Subject	Artificial Intelligence and Expert Systems
Case Study	Smart Healthcare Diagnosis System
Assessment Coverage	All 9 Evaluation Criteria

CERTIFICATE

This is to certify that the project report titled “Smart Healthcare Diagnosis System – Knowledge Representation, Logical Inference and Rule-Based Reasoning” has been prepared as an academic case study demonstrating propositional logic, first-order logic, knowledge base design, unification, resolution, forward chaining, backward chaining, system architecture, and prototype implementation.

Role	Signature / Date
Student	_____Sk. Fawaz Ahamad / 192425390_____
Faculty	_____Dr_M. Prakash_____

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty and institution for providing the opportunity to study artificial intelligence and expert-system reasoning through a practical healthcare case study. This report helped connect formal logic with knowledge engineering and transparent rule-based decision making.

ABSTRACT

The Smart Healthcare Diagnosis System is a rule-based artificial intelligence prototype designed to infer a possible condition from a small set of patient symptoms. The case contains three initial facts about Ravi—fever, cough and body pain—and three rules that derive flu, viral infection and the recommendation to rest. The report formalizes the same knowledge using propositional logic and first-order logic, constructs a knowledge base, demonstrates unification and resolution, and applies both forward and backward chaining. A compact prototype is also evaluated to verify that the expected inference sequence is produced. The approach is intentionally deterministic and explainable: every derived conclusion can be traced to a rule and supporting facts.

TABLE OF CONTENTS

No.	Section
1	Problem Analysis and Domain Understanding
2	Knowledge Engineering and Knowledge Base Design
3	Propositional Logic Representation
4	First-Order Logic Representation
5	Unification and Resolution
6	Forward and Backward Chaining
7	Inference and Reasoning Accuracy
8	System Architecture / Representation Diagram
9	Prototype Implementation and Results
10	Conclusion and Future Scope
11	References

1. PROBLEM ANALYSIS AND DOMAIN UNDERSTANDING

Healthcare diagnosis is a knowledge-intensive task in which observations such as symptoms must be related to possible conditions and recommendations. In this case study, the hospital wants a simple AI system that can reason from patient symptoms using explicit logical rules.

1.1 Problem Statement

Given a patient named Ravi with fever, cough and body pain, determine what conclusions can be inferred from the supplied knowledge. The system must identify whether Ravi may have flu, whether flu together with body pain implies viral infection, and whether the resulting condition leads to a recommendation to rest.

1.2 Domain Objects and Predicates

Symbol / Predicate	Meaning	Example
Fever(x)	x has fever	Fever(Ravi)
Cough(x)	x has cough	Cough(Ravi)
BodyPain(x)	x has body pain	BodyPain(Ravi)
Flu(x)	x may have flu	Flu(Ravi)
ViralInfection(x)	x may have viral infection	ViralInfection(Ravi)
Rest(x)	x should be advised to rest	Rest(Ravi)

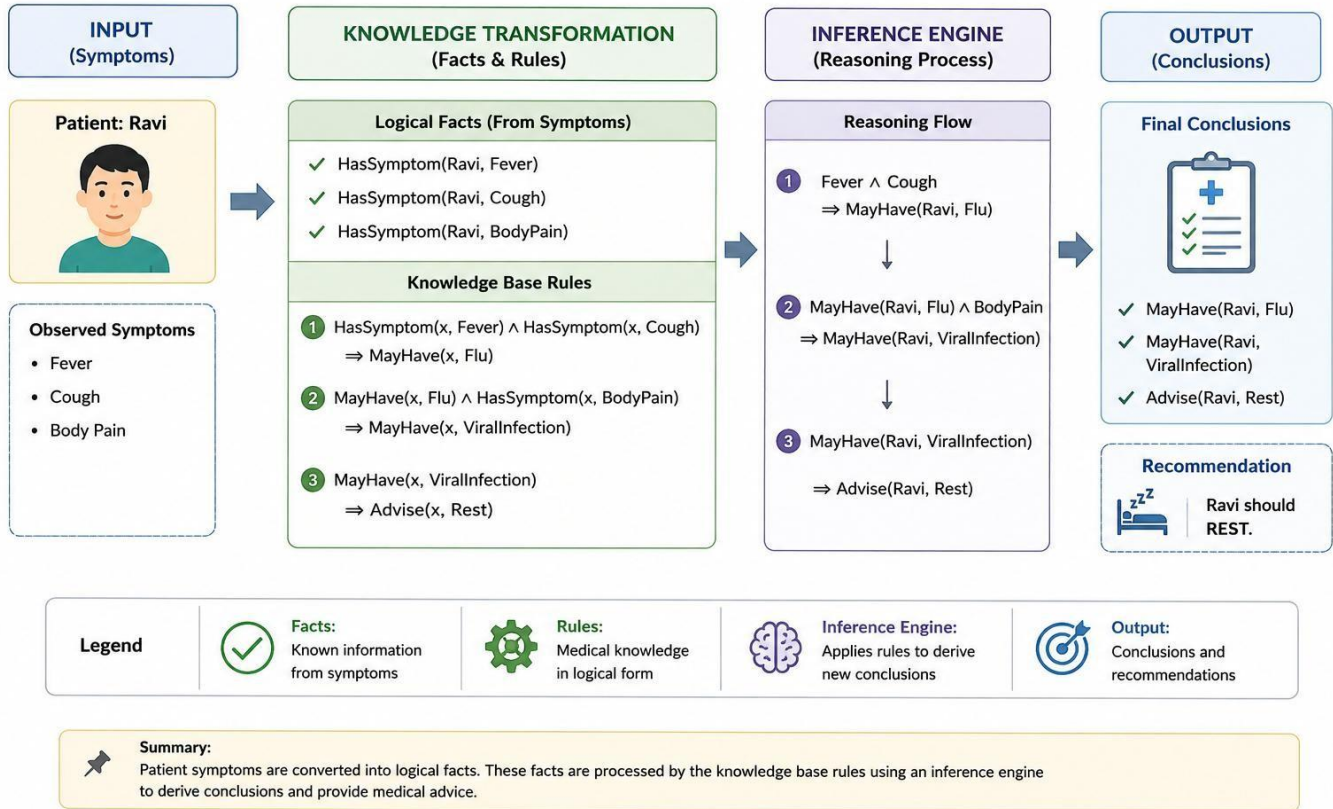
1.3 Given Knowledge

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

1.4 Objectives

1. Represent the facts and rules formally.
2. Build a reusable knowledge base.
3. Demonstrate substitution through unification.
4. Derive conclusions using forward and backward reasoning.
5. Show one resolution proof.
6. Verify the final result using a prototype.

Figure 1. Problem domain: symptoms are transformed into logical facts and conclusions.

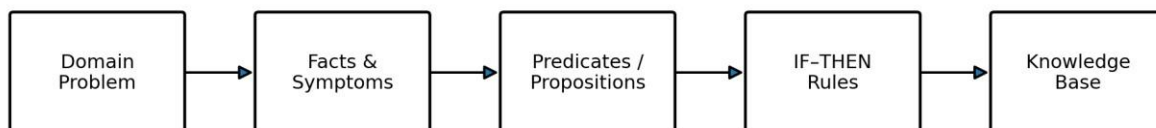


2. KNOWLEDGE ENGINEERING AND KNOWLEDGE BASE DESIGN

Knowledge engineering converts domain knowledge into a representation that an inference engine can process. A rule-based expert system commonly separates the knowledge base from the inference mechanism, allowing rules and facts

?

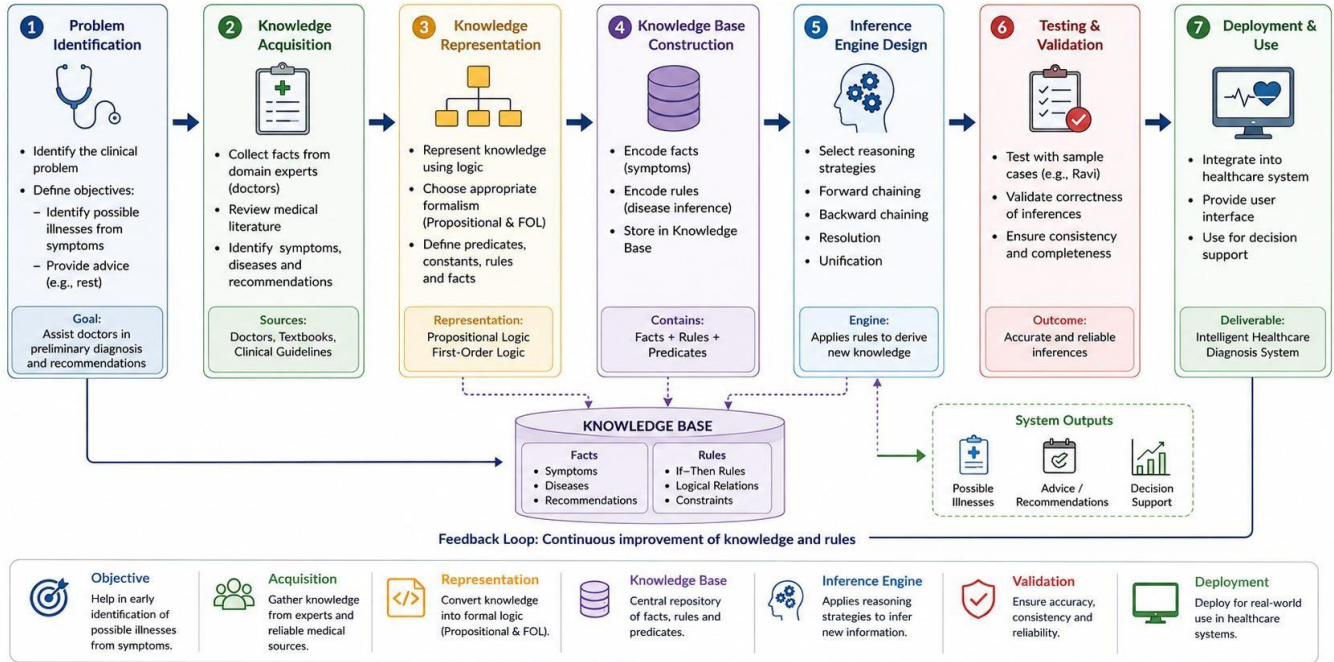
Knowledge Engineering Workflow



Translate observations into formal symbols, then encode relationships as reusable rules.

Figure 2. Knowledge engineering workflow for the healthcare case study.

Figure 2. Knowledge engineering workflow for the healthcare case study.



2.1 Knowledge Base Components

Component	Content in this system	Purpose
Fact Base	Fever(Ravi), Cough(Ravi), BodyPain(Ravi)	Stores observed symptoms.
Rule Base	R1, R2, R3	Stores domain relationships.
Working Memory	Initial + derived facts	Tracks the current reasoning state.
Inference Engine	Forward/backward chaining + resolution	Derives conclusions.
Explanation Layer	Rule trace and proof steps	Shows why a conclusion was reached.

2.2 Production Rules

$$\mathbf{R1: Fever(x) \wedge Cough(x) \rightarrow Flu(x)}$$

$$\mathbf{R2: Flu(x) \wedge BodyPain(x) \rightarrow ViralInfection(x)}$$

$$\mathbf{R3: ViralInfection(x) \rightarrow Rest(x)}$$

These rules are monotonic for this case: newly derived facts are added to the knowledge state and are not used to retract earlier facts. This makes the inference trace easy to inspect.

3. PROPOSITIONAL LOGIC REPRESENTATION

Propositional logic represents complete statements as propositions that are either true or false. Because the case concerns one named patient, each symptom or conclusion can be represented as a proposition.

3.1 Proposition Symbols

Symbol	Proposition
F	Ravi has fever.
C	Ravi has cough.
B	Ravi has body pain.
L	Ravi may have flu.
V	Ravi may have viral infection.
R	Ravi should rest.

3.2 Facts

F = True

C = True

B = True

3.3 Rules

$(F \wedge C) \rightarrow L$

$(L \wedge B) \rightarrow V$

$V \rightarrow R$

3.4 Propositional Derivation

8. F and C are true, therefore L follows by Rule 1.
9. L and B are now true, therefore V follows by Rule 2.
10. V is true, therefore R follows by Rule 3.

Hence the propositional representation proves that Ravi may have flu, may have a viral infection, and should be advised to rest, under the assumptions encoded by the given rules.

Representation Level	Expression
Initial state	$F \wedge C \wedge B$
After R1	$F \wedge C \wedge B \wedge L$
After R2	$F \wedge C \wedge B \wedge L \wedge V$
After R3	$F \wedge C \wedge B \wedge L \wedge V \wedge R$

4. FIRST-ORDER LOGIC REPRESENTATION

First-Order Logic (FOL) is more expressive than propositional logic because it represents individuals, predicates and variables. This allows the same rules to apply to any patient rather than only to Ravi.

4.1 Constants and Predicates

- Constant: Ravi
- Unary predicates: Fever(x), Cough(x), BodyPain(x), Flu(x), ViralInfection(x), Rest(x) • Variable: x, representing an arbitrary patient

4.2 FOL Facts Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

4.3 Universal Rules

$\forall x [(Fever(x) \wedge Cough(x)) \rightarrow Flu(x)]$

$\forall x [(Flu(x) \wedge BodyPain(x)) \rightarrow ViralInfection(x)]$

$\forall x [ViralInfection(x) \rightarrow Rest(x)]$

4.4 Why FOL is Better for a Reusable System

- The rules are general and can be applied to Ravi, Asha, John or any other patient.
- Variables support matching and substitution during inference.
- Predicates make the semantic meaning of symptoms explicit.
- The same knowledge base can handle multiple patient records.

4.5 FOL Inference

Substituting $x = \text{Ravi}$ into the first universal rule gives $\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi}) \rightarrow \text{Flu}(\text{Ravi})$. Since both antecedents are facts, $\text{Flu}(\text{Ravi})$ is derived. The same substitution then enables the second and third rules, producing $\text{ViralInfection}(\text{Ravi})$ and $\text{Rest}(\text{Ravi})$.

5. UNIFICATION AND RESOLUTION

5.1 Unification

Unification finds substitutions that make two logical expressions identical. It is central to FOL inference because a general rule containing a variable must be matched with a specific patient fact.

Expression 1	Expression 2	Substitution θ	Unified result
$\text{Fever}(x)$	$\text{Fever}(\text{Ravi})$	$\{x/\text{Ravi}\}$	$\text{Fever}(\text{Ravi})$
$\text{Cough}(x)$	$\text{Cough}(\text{Ravi})$	$\{x/\text{Ravi}\}$	$\text{Cough}(\text{Ravi})$
$\text{ViralInfection}(x)$	$\text{ViralInfection}(\text{Ravi})$	$\{x/\text{Ravi}\}$	$\text{ViralInfection}(\text{Ravi})$

For example, the rule $\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$ unifies with the facts $\text{Fever}(\text{Ravi})$ and $\text{Cough}(\text{Ravi})$ using $\theta = \{x/\text{Ravi}\}$. The rule therefore produces $\text{Flu}(\text{Ravi})$.

5.2 Resolution

Resolution proves a query by converting implications into clauses and resolving complementary literals. For the selected query $\text{Rest}(\text{Ravi})$, first consider the rule $\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$. Its clause form is $\neg \text{ViralInfection}(x) \vee \text{Rest}(x)$.

With the fact $\text{ViralInfection}(\text{Ravi})$, the complementary literals resolve.

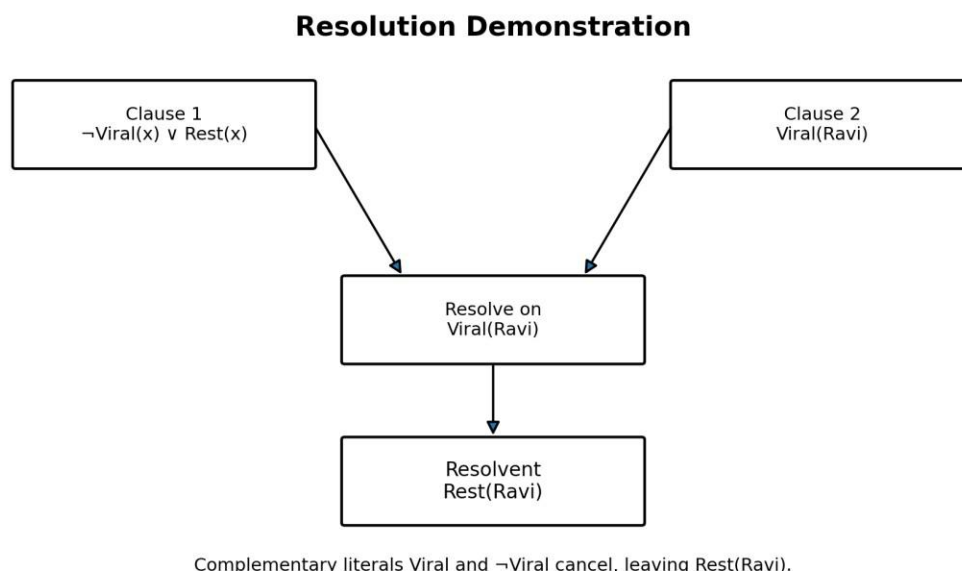


Figure 3. Resolution proof for $\text{Rest}(\text{Ravi})$.

5.3 Clause Form

R3: $\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Clause: $\neg \text{ViralInfection}(x) \vee \text{Rest}(x)$

Fact: $\text{ViralInfection}(\text{Ravi})$

Resolvent: $\text{Rest}(\text{Ravi})$

The resolution step is valid because the positive literal $\text{ViralInfection}(\text{Ravi})$ and the negative literal $\neg \text{ViralInfection}(\text{Ravi})$ are complementary after applying the substitution $x = \text{Ravi}$.

6. FORWARD AND BACKWARD CHAINING

6.1 Forward Chaining

Forward chaining is data-driven reasoning. It starts with known facts, finds rules whose antecedents are satisfied, and adds their conclusions to working memory. This process continues until no new fact can be derived or the desired conclusion is reached. [\[1\] cite\[2\] turn0search2\[2\] turn0search29\[2\]](#) | |

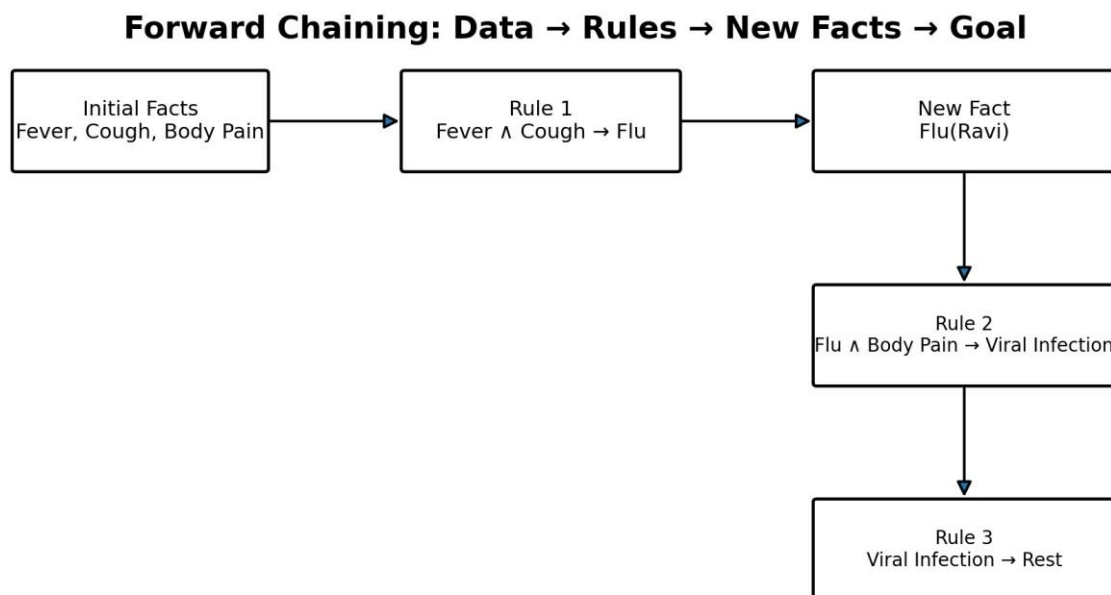


Figure 4. Forward-chaining derivation for Ravi.

11. Initial facts: $\text{Fever}(\text{Ravi})$, $\text{Cough}(\text{Ravi})$, $\text{BodyPain}(\text{Ravi})$.
12. R1 matches Fever and $\text{Cough} \rightarrow$ derive $\text{Flu}(\text{Ravi})$.
13. R2 matches Flu and $\text{BodyPain} \rightarrow$ derive $\text{ViralInfection}(\text{Ravi})$.
14. R3 matches $\text{ViralInfection} \rightarrow$ derive $\text{Rest}(\text{Ravi})$.

6.2 Backward Chaining

Backward chaining is goal-driven. Instead of generating every possible conclusion, the system starts with a target such as Rest(Ravi), finds a rule that can produce it, and recursively asks whether the rule's premises can be established.

Backward Chaining: Goal → Required Conditions → Known Facts

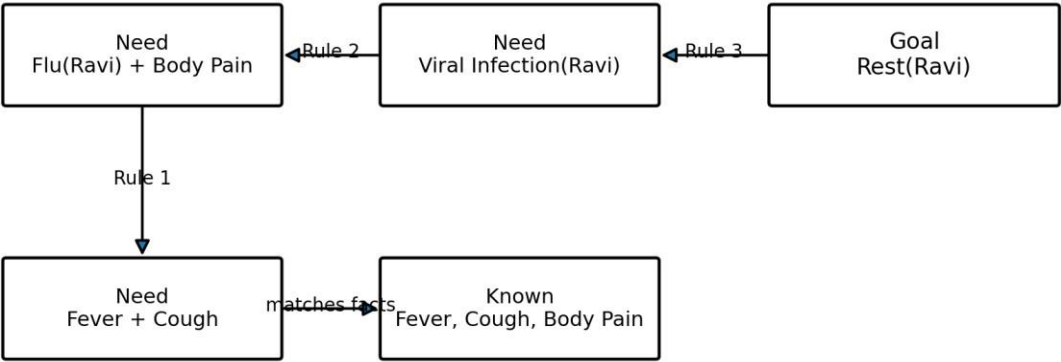


Figure 5. Backward-chaining proof search for Rest(Ravi).

Starting from Rest(Ravi), the system selects R3 and requires ViralInfection(Ravi). R2 then requires Flu(Ravi) and BodyPain(Ravi). R1 requires Fever(Ravi) and Cough(Ravi). All required base facts are available, so the goal is proven.

6.3 Comparison

Factor	Forward Chaining	Backward Chaining
Direction	Facts → conclusion	Goal → required facts
Style	Data-driven	Goal-driven
Best suited for	Multiple possible conclusions	A specific query
Case result	Derives Flu, Viral Infection, Rest	Proves Rest directly
Explainability	Trace of fired rules	Proof path from goal to facts

7. INFERENCE AND REASONING ACCURACY

Reasoning accuracy is evaluated by checking whether every derived statement follows from a satisfied rule and whether no unsupported conclusion is introduced. In this controlled case, all three rules are deterministic and the prototype reaches the expected final state.

Final Inference Tree

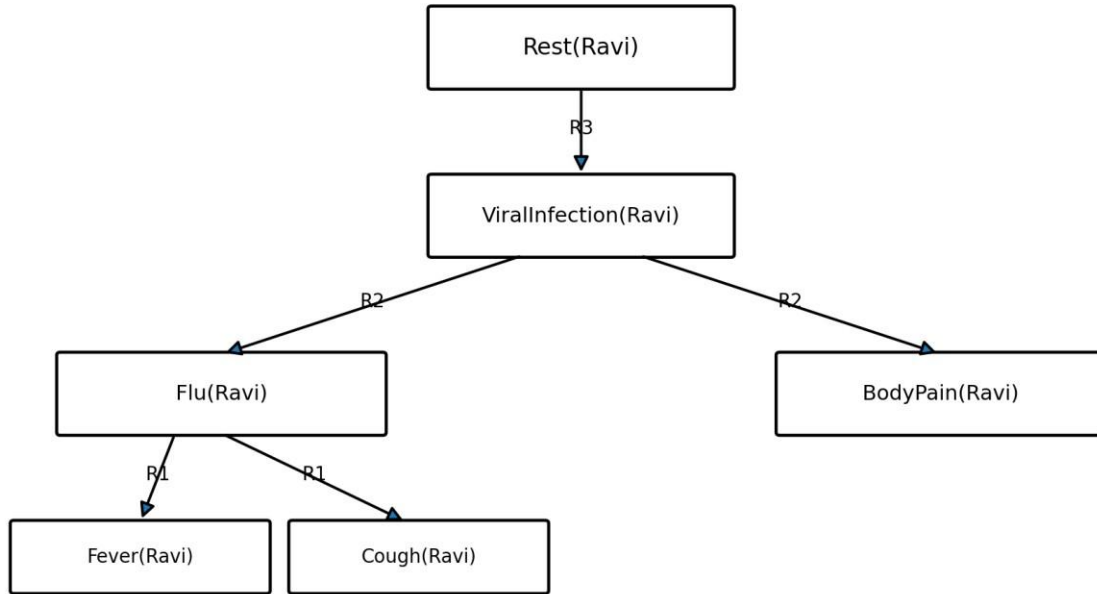


Figure 6. Inference tree showing the complete support chain.

7.1 Step-by-Step Verification

Step	Rule	Required facts	Conclusion	Status
1	R1	Fever + Cough	Flu	Valid
2	R2	Flu + Body Pain	Viral Infection	Valid
3	R3	Viral Infection	Rest	Valid

7.2 Query Verification

- Query: Does Ravi have flu? → Yes, according to the supplied rule set.
- Query: Does Ravi have viral infection? → Yes, because Flu(Ravi) and BodyPain(Ravi) are established.
- Query: Should Ravi rest? → Yes, because ViralInfection(Ravi) is established.

7.3 Reasoning Limitations

- The system reasons only from the supplied facts and rules.
- The phrase “may have” is treated as a logical conclusion in this educational model; it is not a probability.
- No medical history, laboratory tests, severity, uncertainty or differential diagnosis is modeled.
- Therefore, the output is an academic reasoning result and must not be treated as a clinical diagnosis.

8. SYSTEM ARCHITECTURE / REPRESENTATION DIAGRAM

The proposed architecture separates user observations, the knowledge base and the inference engine. This separation is a standard design principle in knowledge-based expert systems and supports maintainability and explainability.

[cite](#) [turn0search1](#) [turn0search5](#)

Smart Healthcare Diagnosis System — Rule-Based Architecture

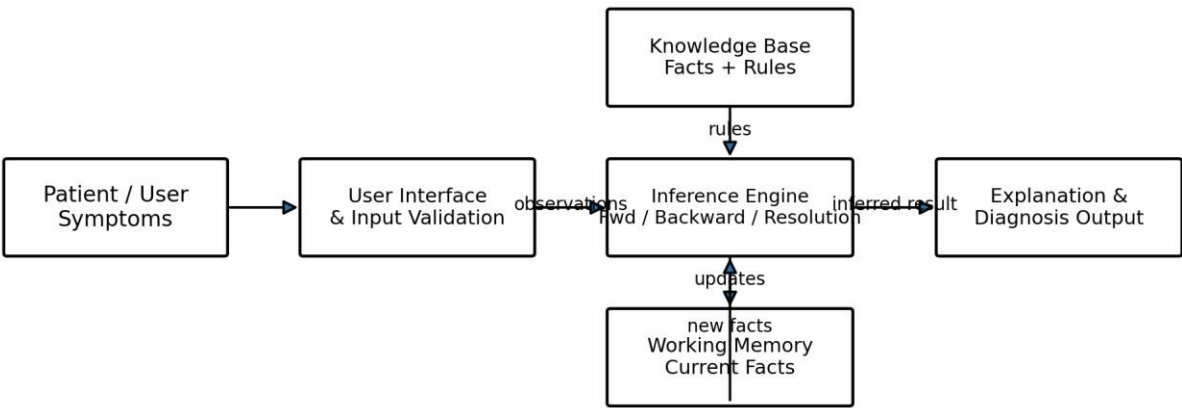


Figure 7. Complete system architecture for the Smart Healthcare Diagnosis System.

8.1 Component Description

Component	Responsibility
User Interface	Accepts patient symptoms and displays conclusions.
Input Validation	Checks that symptom values are valid and complete.
Knowledge Base	Stores facts and general IF–THEN rules.
Working Memory	Maintains facts relevant to the current patient.
Inference Engine	Matches facts with rules and derives new facts.
Explanation Module	Shows the rules and facts supporting a conclusion.
Output Layer	Reports inferred condition/recommendation with a disclaimer.

8.2 End-to-End Representation Flow

- 15. Patient symptoms are entered.
- 16. Symptoms are converted to predicates.
- 17. Predicates become facts in working memory.
- 18. Rules are matched using unification.
- 19. Forward or backward reasoning derives conclusions.

- 20. Resolution can independently verify a selected query.
- 21. The explanation module presents the inference trace.
- 22. The output is shown as an educational recommendation.

9. PROTOTYPE IMPLEMENTATION AND RESULTS

A lightweight Python-style rule engine can implement the case without requiring a machine-learning model. The prototype stores facts in a set, checks whether each rule's premises are satisfied, and adds the conclusion whenever a rule fires.

9.1 Prototype Pseudocode

```
facts = {
    "Fever(Ravi)",
    "Cough(Ravi)",
    "BodyPain(Ravi)"
}
rules = [
    ({ "Fever(Ravi)", "Cough(Ravi)" }, "Flu(Ravi)"),
    ({ "Flu(Ravi)", "BodyPain(Ravi)" }, "ViralInfection(Ravi)"),
    ({ "ViralInfection(Ravi)" }, "Rest(Ravi)")
]
repeat:
    for premises, conclusion in rules:
        if premises ⊆ facts:
            add conclusion to facts until no new fact is added
print(facts)
```

9.2 Execution Result

Prototype Execution Results

Step	Matched condition	Derived fact
1	Fever(Ravi) ∧ Cough(Ravi)	Flu(Ravi)
2	Flu(Ravi) ∧ BodyPain(Ravi)	ViralInfection(Ravi)
3	ViralInfection(Ravi)	Rest(Ravi)

Final knowledge state includes Fever, Cough, Body Pain, Flu, Viral Infection and Rest.

Figure 8. Prototype execution trace and final derived facts.

9.3 Result Interpretation

Input / Condition	Observed Result
Fever(Ravi)	Available as initial fact
Cough(Ravi)	Available as initial fact
BodyPain(Ravi)	Available as initial fact
Flu(Ravi)	Derived by R1
ViralInfection(Ravi)	Derived by R2
Rest(Ravi)	Derived by R3
Final query: Rest(Ravi)	Proved

9.4 Prototype Performance

For this small knowledge base, inference is computationally inexpensive. A simple forward-chaining implementation repeatedly scans a small number of rules until a fixed point is reached. In larger systems, indexing rules by predicates and using efficient pattern matching would reduce unnecessary rule checks.

10. CONCLUSION AND FUTURE SCOPE

The Smart Healthcare Diagnosis System demonstrates how a small set of patient observations can be transformed into formal knowledge and processed through symbolic reasoning. Propositional logic provides a compact representation for a single case, while first-order logic generalizes the rules to arbitrary patients. Unification connects general predicates with patient-specific facts, resolution provides a formal proof mechanism, and forward/backward chaining provide complementary inference strategies.

10.1 Final Inference

$$\text{Fever(Ravi)} \wedge \text{Cough(Ravi)} \rightarrow \text{Flu(Ravi)}$$

$$\text{Flu(Ravi)} \wedge \text{BodyPain(Ravi)} \rightarrow \text{ViralInfection(Ravi)}$$

$$\text{ViralInfection(Ravi)} \rightarrow \text{Rest(Ravi)}$$

Therefore, within the supplied knowledge base, the final inferred recommendation is: Ravi should be advised to rest because the reasoning chain derives ViralInfection(Ravi). This is a logical inference produced by the educational rule set, not a medically validated diagnosis.

10.2 Future Scope

- Add uncertainty using probability or fuzzy logic rather than treating every rule as certain.
- Add laboratory observations and medical history as additional predicates.

- Use a larger clinical knowledge base maintained by qualified medical experts.
- Add conflict resolution when multiple rules produce competing conclusions.
- Develop a graphical user interface with an explanation panel.
- Add test cases for missing symptoms, contradictory facts and multiple patients.
- Evaluate precision, recall and coverage against a clinically validated reference set before any real-world use.

11. REFERENCES

- Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
- Nilsson, N. J., Artificial Intelligence: A New Synthesis, Morgan Kaufmann.
- S. J. Russell and P. Norvig, chapters on knowledge representation, logical agents and inference.
- ScienceDirect Topics, “Rule-Based System” — overview of production-rule systems and inference components.
- ScienceDirect Topics, “Expert System” — architecture, knowledge base, inference engine and explanation subsystem.
- IEEE Technology Navigator, “Decision and Expert Systems” — rule-based expert-system architecture and chaining concepts.

Evaluation Marks

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.

Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.
Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.

Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.
---	----	---	--	--	--